

Amortized Analysis

Algorithms COMP.5030 — structured from NuMarkdown-8B-Thinking OCR (corrected)

Prof. Hugo A. Akitaya

[CLRS ch. 17]

1 Introduction

Slide 1: Title

Amortized Analysis

Prof. Hugo A. Akitaya [CLRS ch. 17]

Slide 2: Idea

- **Naïve analysis is not tight:** a simple “worst-case every time” estimate can make an algorithm look slower than it really is.
- **Algorithms do many operations:** an algorithm is made of lots of small steps.
- **Some steps are expensive, most are cheap:** occasionally you do a slow/heavy operation, but most of the time you do quick/light ones. If you pretend *every* step is the expensive one, you get an overly pessimistic upper bound.
- **Lazy approach:** don’t do big work immediately; postpone it until it’s truly needed. This often reduces how often you pay the big cost.
- **Amortization (amortized analysis):** instead of looking at the cost of one operation alone, look at the **total cost over many operations** and spread it out. This shows the **average cost per operation** is usually small, even if a few operations are very expensive.
- **Example idea:** resizing a dynamic array is expensive sometimes, but because it happens rarely, the **average** cost per insert is still small.

2 Example 1: Incrementing a Binary Counter

Slide 3: The data structure

- Binary counter $A[1..k]$ with k bits stores a number x with $0 \leq x \leq 2^k - 1$ ($x = 0$ at the beginning):

$$x = \sum_{i=1}^k A[i] \cdot 2^{i-1}.$$

- Each bit has a different *place value*; the array stores the number **least significant bit first**:

Array position	Bit	Value
$A[1]$	0 or 1	$2^0 = 1$
$A[2]$	0 or 1	$2^1 = 2$
$A[3]$	0 or 1	$2^2 = 4$
$A[4]$	0 or 1	$2^3 = 8$

Example: $A = [1, 1, 0, 1]$ gives $x = 1(2^0) + 1(2^1) + 0(2^2) + 1(2^3) = 1 + 2 + 0 + 8 = 11$. Visually one would write 1011_2 , but the array is $[1, 1, 0, 1]$.

- Why 2^k values? Each of the k bits has 2 possibilities, so there are 2^k combinations; starting at 0, the largest value is $2^k - 1$. (3 bits: 000, 001, ..., 111 represent 0, 1, ..., 7.)
- How do we add one to x ? Binary addition has a **carry**, just like $199 + 1 = 200$ in decimal. Start with the least significant bit $A[1]$:
 - **Case 1:** $A[1] = 0$. Then $0 + 1 = 1$: flip it and finish (only *one bit* changes), e.g. $[0, 1, 0, 0] \rightarrow [1, 1, 0, 0]$.
 - **Case 2:** $A[1] = 1$. Then $1 + 1 = 10_2$: set the bit to 0 and carry the 1 to the next bit, e.g. $[1, 0, 0, 0] \rightarrow [0, 1, 0, 0]$.
 - If the next bit is also 1, the carry keeps travelling: $[1, 1, 1, 0]$ (value 7) becomes $[0, 0, 0, 1]$ (value 8) — four bits modified.
- Pseudocode:

```

INCREMENT(A):
    i = 1
    while i <= k and A[i] == 1:
        A[i] = 0
        i = i + 1
    if i <= k:
        A[i] = 1

```

- Why it stops: the first 0 encountered becomes 1; the 1s before it become 0s; everything after is unchanged: $\underbrace{11\dots 1}_\text{become 0} \underbrace{0}_\text{becomes 1} \rightarrow \underbrace{00\dots 0}_\text{carry} 1$.
- Two questions: worst-case and best-case cost *per operation*?

Slide 4: Demo

Binary counter increment demo (interactive animation).

Slide 5: Worst and best case

- Best case: $A[1] = 0$ already — only one bit changes, so cost $O(1)$ (essentially one bit update), e.g. $[0, 1, 1, 0, 1] \rightarrow [1, 1, 1, 0, 1]$.
- Worst case: all k bits are 1 (e.g. $[1, 1, 1, 1, 1] = 31$; adding 1 gives $32 = 100000_2$, which wraps to 00000_2 with only 5 bits) $\rightarrow$ the carry propagates through all k bits, cost $\Theta(k)$. (slide: $\Omega(k)$)

- Odometer intuition (decimal): 0009 $\rightarrow$ 0010 changes two digits, 0010 $\rightarrow$ 0011 changes one, but 0099 $\rightarrow$ 0100 changes three and 0999 $\rightarrow$ 1000 changes four. The binary counter behaves the same way: most operations are cheap, occasionally one is expensive — **worst-case cost** $\neq$ **amortized cost**.
- Cost of performing n operations:
 - Naïve analysis: $O(nk)$.
 - Intuition: “good cases” happen much more often than “bad cases”. Watching consecutive increments from 0, the numbers of bits changed are 1, 2, 1, 3, 1, 2, 1, 4, $\dots$ — most increments change only one or two bits.
 - **Amortized:** $O(n)$ total cost.
- Three answers to remember:

Analysis	Cost
Best case for one increment	$O(1)$
Worst case for one increment	$O(k)$
Amortized cost over many increments	$O(1)$

Aggregate analysis

Slide 6: Aggregate analysis: questions

- Strategy: do *not* analyze each operation in isolation — instead compute the **total** cost of a sequence of n operations, then divide by n .
- The cost of one increment equals the number of bits that flip in it. So ask, for each bit: how many times does it flip over n increments?
- Sum up the total cost:
 - Bit $A[1]$ changes how many times? (Every operation — it flips $1 \rightarrow 0$ or $0 \rightarrow 1$ on *every* increment, so $\approx n$ times.)
 - How about $A[2]$? (pair discussion) — *answer:* it flips every 2nd increment, i.e., whenever $A[1]$ was 1 and carries into it: increments 2, 4, 6, $\dots$, so $\approx n/2$ times.
 - Following the same pattern: $A[3]$ flips every 4th increment ($\approx n/4$), $A[4]$ every 8th ($\approx n/8$), and in general $A[i]$ every 2^{i-1} -th increment ($\approx n/2^{i-1}$).

Slide 7: Aggregate analysis: solution

- Why does $A[i]$ flip only then? An increment flips $A[i]$ only when the carry reaches it, i.e., only when $A[1..i-1]$ are all 1s before the increment. Among n consecutive increments starting from 0, that condition holds for the i th bit at most $\lfloor n/2^{i-1} \rfloor$ times (the counter wraps through its low i bits once every 2^{i-1} increments).
- Per-bit flip counts over n operations:

$$A[1] : n, \quad A[2] : \approx n/2, \quad A[3] : \approx n/4, \quad A[4] : \approx n/8, \quad \dots$$

Concretely (8 increments from 0): bits flipped per increment are 1, 2, 1, 3, 1, 2, 1, 4 — a total of $15 = 8 + 4 + 2 + 1$ flips, matching $n + n/2 + n/4 + n/8$.

- Total cost (summing over all k bits):

$$\sum_{i=1}^k \left\lfloor \frac{n}{2^{i-1}} \right\rfloor \leq n \sum_{j=0}^{\infty} \frac{1}{2^j} = n \cdot \frac{1}{1 - \frac{1}{2}} = 2n \quad (\text{geometric series}).$$

Since $1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots < 2$, the total cost of n increments is $< 2n = O(n)$, *regardless of k* .

- Amortized cost = total cost / number of operations = $2n/n = 2 = O(1)$.
- Takeaway: even though a *single* increment can cost $\Theta(k)$, the *average* cost over any sequence of increments is $O(1)$ — the expensive ones are so rare that the geometric series absorbs them.

Accounting method

Slide 8: Accounting method

- Main idea: **charge cheap operations a little extra, save that extra amount, and use the savings to pay for expensive operations later.**
- For operation i : c_i = **actual cost** (what it really does), $\hat{c}_i$ = **amortized cost** (what we assign/charge). They don't have to be equal, e.g. cheap ops $c_i = 1$ charged $\hat{c}_i = 2$; an expensive op $c_i = 4$ still charged only $\hat{c}_i = 2$.
- Bank-account intuition: amortized cost = money you charge; actual cost = money you actually spend. If $\hat{c}_i > c_i$, the leftover becomes **credit**; later, when $c_i > \hat{c}_i$, accumulated credit covers the difference.
- Intuition: leave coins on the data structure to pay later for expensive operations. Each element can have a coin sitting on it — coins aren't real money, just a bookkeeping device proving we have enough saved credit.
- Decide the amortized cost $\hat{c}_i$ of the i th operation so that
 - $\hat{c}_i \geq c_i$ for cheap operations (saving for a rainy day; the extra $\hat{c}_i - c_i$ becomes credit),
 - expensive operations may have $\hat{c}_i < c_i$ (shortfall paid from savings),
 - $\sum_{i=1}^n \hat{c}_i \geq \sum_{i=1}^n c_i$, i.e. $\sum_{i=1}^n (\hat{c}_i - c_i) \geq 0$: the total charged must be at least the total actual cost — we never owe anything in the end (expensive operations are paid from the reserve in the bank). $\hat{c}_i - c_i$ is exactly the change in the bank balance.
- Caution: you can't pick any charge you like. Example: actual costs 1, 1, 1, 4, 1, 1, 1, 8 total 18, but charging 2 each gives only 16 — not enough. You must *prove* the chosen charge always leaves enough credit.
- Warning: amortized analysis $\neq$ average-case analysis. Average-case involves probabilities over inputs; amortized analysis needs **no randomness** — it holds for *every* sequence of n operations, even the worst possible one.

Slide 9: Accounting method: binary counter

- Charge $\hat{c}_i = 2$ coins for each increment. Why 2? Every increment (1) flips t ones $\rightarrow$ zeros and (2) flips one 0 $\rightarrow$ 1; the newly created 1 can hold a coin.

- Invariant: every bit valued 1 has a coin stored on it (so credit Φ_i = number of saved coins = number of 1s ≥ 0 at all times — we never borrow imaginary money from the future).
 - $A[1]$ changes $0 \rightarrow 1$: pay 1 coin for the change and store 1 coin on $A[1]$.
 - $A[1]$ changes $1 \rightarrow 0$: let $A[j]$ be the bit that changes $0 \rightarrow 1$, so $A[1..j-1]$ are all 1s that become 0. Use the $j-1$ stored coins to flip $A[1..j-1]$; from the 2 new coins, pay 1 to change $A[j]$ and store 1 on $A[j]$.
- Walkthrough (charging 2 each time): $0000 \rightarrow 0001$ ($c_1 = 1$: spend 1, save 1 on the new 1); $0001 \rightarrow 0010$ ($c_2 = 2$: 2 charged + the stored coin on the old 1 pay for both flips); $0010 \rightarrow 0011$ ($c_3 = 1$: spend 1, save 1); $0011 \rightarrow 0100$ ($c_4 = 3 > 2$: the coins stored on the two 1s pay for the carry — the expensive operation was **prepaid by earlier cheap ones**).
- General increment with t consecutive 1s: actual cost $c_i = t + 1$ (can be $O(k)$), but the t stored coins pay for the t bits flipping $1 \rightarrow 0$, and the 2 new coins pay for the new 1 (1 to spend, 1 to save). All work is paid for, so $\hat{c}_i = 2 = O(1)$ safely.
- Every $1 \rightarrow 0$ flip is paid by a coin stored when that bit earlier went $0 \rightarrow 1$ — expensive carries never come out of nowhere.
- The bank is never negative (the number of 1s is nonnegative), hence

$$\sum_{i=1}^n \hat{c}_i \geq \sum_{i=1}^n c_i.$$

- Big picture: actual cost $c_i = t + 1 = O(k)$; amortized cost $\hat{c}_i = 2 = O(1)$; total over n increments $\sum c_i = O(n)$, so the average is $O(n)/n = O(1)$.
- Exam checklist: (1) give each op an amortized charge $\hat{c}_i$; (2) cheap ops overcharged; (3) the difference becomes credit; (4) expensive ops use the credit; (5) credit never negative; (6) show $\sum \hat{c}_i \geq \sum c_i$; (7) for the binary counter, $\hat{c}_i = 2$ gives $O(1)$ amortized even though one increment can cost $O(k)$.

Potential method

Slide 10: Potential method

- The data structure D changes with each operation: D_0 is the initial state, D_i the state after the i th operation.
- Define a potential function $\Phi : D \rightarrow \mathbb{R}$.
- Common (not required) conditions: $\Phi(D_0) = 0$ and $\Phi(D_i) \geq 0$ for all i .
- Required: $\Phi(D_n) \geq \Phi(D_0)$.
- Amortized cost of the i th operation:

$$\hat{c}_i = c_i + \Delta\Phi = c_i + \Phi(D_i) - \Phi(D_{i-1}).$$

Slide 11: Potential method: binary counter

- Define $\Phi(D)$ = number of 1s in D (number of coins in the bank); $\Phi(D_0) = 0$, $\Phi(D_i) \geq 0$.

- Let t_i be the number of 1s turned to 0 by the i th increment. The real cost is $c_i = t_i + 1$, and the potential change is $\Delta\Phi = 1 - t_i$:

$$\hat{c}_i = c_i + \Delta\Phi = (t_i + 1) + (1 - t_i) = 2.$$

Slide 12: Demo

Binary counter demo continued (interactive animation).

3 Example 2: Dynamic Array — Push Only

Slide 13: Dynamic array (push only)

- A **dynamic array** is an array that can **grow** when you keep adding (“push”) items, like a **stack**.
- Goal: implement a stack with an array.
- Problem with normal arrays: you must choose the size early. Arrays are classically allocated statically (maybe before we know the size of the data):
 - Allocate too much space $\rightarrow$ wasted space (wasted memory).
 - Allocate too little space $\rightarrow$ wasted time: you must create a bigger array and copy everything into it.
- Idea: keep the array **about half full** — maintain the load factor at a constant (around 1/2?). That way you don’t waste too much space, and you don’t resize too often.
- Operations: PUSH — worst case? (pair discussion) — usually push is **fast** ($O(1)$: just place the element); worst case, when the array is full, push is **slow** because of copying ($O(num)$). But doubling makes this rare, so the average push is still fast (amortized $O(1)$).

Slide 14: Insert pseudocode

- What the variables mean: A = the underlying array (storage); $size$ = the **capacity** of A (how many slots exist), initially 0; num = the **number of elements currently stored** (how many slots are used), initially 0.
- PUSH(x) in simple steps:
 - If this is the first insert ($size = 0$, edge case): create an array (often $size = 1$).
 - If the array is **full** ($num = size$): make a **new array** with **double** capacity $2 \cdot size$, **copy** all old elements into it, and replace A with the new array.
 - Put the new element in the next free spot: $A[num] = x$.
 - Increase num by 1 (and update $size$ if a resize happened).
- Cost: cheap push $O(1)$; push with resize $O(num)$ — but resizes happen rarely, so amortized $O(1)$.

Slide 15: Aggregate method

- Main idea: most insertions are cheap, occasionally one is expensive, but averaged across many insertions each is still $O(1)$.
- What happens when we push? Starting from capacity 1, the array doubles when full:

```
capacity 1: [A]
push B -> resize: [A, B]                (capacity 2)
push C -> resize: [A, B, C, _]          (capacity 4)
push D -> no resize: [A, B, C, D]
push E -> resize: [A, B, C, D, E, _, _, _] (capacity 8)
```

- Perform n inserts. Two kinds of pushes:
 - **Cheap push** (free space available): just add the item at the tail, e.g. $[A, B, _, _] \rightarrow [A, B, C, _]$; cost $O(1)$ — happens often.
 - **Expensive push** (array full): create a larger array and copy everything, e.g. copy $[A, B, C, D]$ into a new array of capacity 8; cost $O(i)$ because there may be i elements to copy — happens only if $i = 2^k$ for some integer k (resizes occur at sizes 1, 2, 4, 8, ...).
- Aggregate method: don't worry about the cost of one push — add up the cost of all n pushes. Most pushes cost 1, contributing $O(n)$; the total copying work is $1 + 2 + 4 + 8 + \dots$ (e.g. for 16 pushes: $1 + 2 + 4 + 8 = 15 < 16$).
- Total cost:

$$O(n) + \sum_{k=0}^{\lfloor \log_2 n \rfloor} 2^k \leq O(n) + 2n = O(n).$$

In words: if you add all the copying caused by resizing, the total copying is at most roughly $2n$.

- Average cost per push = $O(n)/n = O(1)$.

Slide 16: Accounting method

- Goal: show that even though resizing is expensive, the *average* cost per push is $O(1)$ using the “coin” (accounting) idea.
- Perform n pushes; charge $\hat{c}_i = 3$ per push:
 - **1 coin** pays for the actual insertion (write the element at the end).
 - The remaining **2 coins** are **saved** to pay for future copying during a resize.
- Two equivalent ways to “store” the saved coins (the difference is only *where* the coins sit; every push still uses 1 coin for insertion):
 - **Version 1 (simplest)**: save 2 coins on *every* element. After 4 pushes (capacity 4): $4 \times 2 = 8$ saved coins.
 - **Version 2 (like the slide)**: save 2 coins only on elements in the *last half* of the array. For capacity 4, last half = positions 3 and 4 $\rightarrow 2 \times 2 = 4$ saved coins (savings concentrated, not spread).

- Invariant (version 2): every element in the last half of the array stores two coins; the bank is always positive — we never spend more coins than we have saved, so we never go into “coin debt.” Example (capacity 8, after resizing from 4): the next four cheap pushes E, F, G, H land in the last half, each storing 2 coins $\rightarrow 4 \times 2 = 8$ saved coins, exactly enough to copy all 8 elements at the next resize. In general, capacity n : last half has $\frac{n}{2}$ elements $\times 2$ coins $= n$ coins for copying the n existing elements.
- **Cheap insertion:** use 1 coin to pay for the push and deposit 2 coins on the new position.
- **Expensive insertion:** by the invariant there are $\frac{n}{2} \cdot 2 = n$ coins stored; use them to pay for the array expansion. We still have the 3 new coins: a cheap push into the new array.
- What happens on the 5th push (array full, resize $4 \rightarrow 8$):
 1. **Resize + copy:** create a new array of size 8 and copy the 4 elements — copying costs 4 operations, paid using the saved coins.
 2. **Insert 5th element:** put it into the new array (cost 1 coin).
 3. **Save again:** from the 3 coins charged, after paying 1, save 2 coins (toward the next resize).
- Key idea: resizes are paid for by coins saved from many earlier cheap pushes, so the average cost stays constant.

Slide 17: Potential method

- Both this and the accounting method prove the same result: pushing into a dynamic array has **amortized cost** $O(1)$, even though resizing is occasionally expensive.
- The potential method is basically the same savings idea, but instead of imaginary coins, the amount saved is represented by a mathematical function, the **potential** Φ . Mentally: $\Phi = \text{saved work/coins for the future}$.
- Define Φ with $\Phi(A_0) = 0$.
- Idea: before an expensive push we have just enough coins, i.e., i coins for the i th push; after a resize the potential is (nearly) depleted. Let $size_i$ denote the array size after the i th operation (i = number of elements after i pushes):

$$\Phi(A_i) = \left(i - \frac{size_i}{2}\right) \cdot 2 = 2i - size_i.$$

- Example: resized to capacity 8 with 4 elements $[A, B, C, D, _, _, _, _]$: $\Phi = 2(4) - 8 = 0$ — immediately after a resize the potential is 0.
- Insert E : $\Phi = 2(5) - 8 = 2$. A cheap push saved exactly 2 coins in the accounting method, and here the potential increases by 2 — *the same idea!* Insert F : $\Phi = 2(6) - 8 = 4$.
- Growth between resizes (capacity 8):

Elements i	Capacity $size_i$	Potential $2i - size_i$
4	8	0
5	8	2
6	8	4
7	8	6
8	8	8

Just before the next resize ($[A, \dots, H]$ full), $\Phi = 8$: enough accumulated “saved work” to pay for moving the 8 existing elements.

- Then the expensive push happens: the array is full, capacity doubles $8 \rightarrow 16$, the accumulated potential accounts for the copying, and after the resize the potential resets (≈ 0) and saving begins again.
- Accounting vs. potential: the accounting method thinks in *coins* (charge 3, spend 1, save 2, use savings for the resize); the potential method thinks in *stored energy* (potential increases with cheap pushes, is consumed by the resize). Saved coins $\approx$ potential.
- Why do this? Looking at one resize alone suggests push is $O(n)$, but that’s misleading: resizing doesn’t happen on every push. By saving a little credit during cheap operations we pay for the rare expensive ones, so the amortized cost per push is $O(1)$.
- Amortized costs: (pair discussion)

Slide 18: Poll: potential of a given array

Poll (in-class)
 What is the potential of the following array?
 Forms: <https://forms.office.com/r/AZbBVrvYEj>

4 Dynamic Array — Push and Pop

Slide 19: Dynamic array (push + pop)

- Main trick: **grow when full, but don’t shrink too quickly when deleting.**
- Reminder: $size$ = capacity of the array; num = number of elements currently stored (e.g. $[A, B, C, _, _, _, _, _]$: $num = 3$, $size = 8$).
- **Load factor** = how full the array is: $\alpha = num/size$. Example $[A, B, C, D, _, _, _, _]$: $\frac{4}{8} = \frac{1}{2} = 50\%$ full.
- Goal: maintain the load factor at least a constant (around how much?).
- Operations:
 - PUSH: same as before — cheap $O(1)$ if there is space; if full, double the array and copy (expensive, but rare).
 - POP: removes the last element, e.g. $[A, B, C, D] \rightarrow [A, B, C, _]$; normally very cheap ($O(1)$). Worst case?
- New problem: capacity 16 but only 2 elements left after many pops $\rightarrow$ a large array holding almost nothing = wasted memory, so at some point we should shrink.

- Pair: what happens if you downsize by half whenever the load factor drops below 1/2? (Threatening: thrashing at the boundary.) Popping drops it slightly below 1/2 $\rightarrow$ shrink; pushing fills it up $\rightarrow$ double; pop $\rightarrow$ shrink; push $\rightarrow$ grow; ... every operation pays for a resize that copies many elements.
- Solution: deliberately leave a **gap** between the two resize conditions — grow when 100% full, shrink only below 25% full:

0%	25%	50%	100%
SHRINK (below 1/4)		GROW (full)	

Bus analogy: when the bus is completely full, get one twice as big; but if a few passengers leave, you don't immediately switch to a smaller bus — you wait until it's very empty (around 1/4 full).

- Now $num_i \neq i$ in general: num_i denotes the number of elements after the i th operation.

Slide 20: Delete pseudocode

- State: array A (initially NIL), integer $size$ (capacity, initially 0), integer num (number of elements, initially 0).
- POP() step by step:
 - **Edge case when** $num = 1$: removing it makes the array empty, so special handling is needed.
 - **Check if the array is too empty:** $num - 1 < \frac{1}{4} \cdot size$, where $num - 1$ = “how many elements will remain after POP”. If that would be less than 25% of capacity, shrink.
 - **Shrink:** create a new array of length $size/2$ and copy the data (e.g. capacity 16 $\rightarrow$ 8; with only 3 elements in a capacity-16 array, $\frac{1}{4} \cdot 16 = 4$, and $3 < 4$, so we shrink to $[A, B, C, _, _, _, _, _]$ — much less wasted memory).
 - **Delete the last element:** $A[num] = \text{NIL}$ clears the position being removed.
 - **Update $size$ and num :** num decreases (one fewer element); $size$ decreases if the array was resized.
- Exam summary: PUSH — if full, **double** the array; POP — if less than 1/4 full, **cut size in half**; the reason for 1/4 instead of 1/2 is **to avoid repeatedly growing and shrinking**.
 $\frac{num}{size}$ = “how full is my array?”

Slide 21: Potential method (push + pop)

- Basic idea: **potential = saved credit for future expensive resizing**. Cheap operations build up enough credit to pay for the occasional expensive resize.
- Terms: num = number of elements actually inside the array; $size$ = total capacity; $\alpha = num/size$ = load factor (how full the array is, e.g. $num = 4$, $size = 8 \Rightarrow \alpha = 1/2$).
- Two expensive situations to prepare for, i.e., potential must work in *both directions*:
 - **Expensive push:** array full $\rightarrow$ double and copy (need credit when too full).
 - **Expensive pop:** array below 1/4 full $\rightarrow$ halve and copy (need credit when too empty).

So: too empty $\rightarrow$ shrink (credit needed) • around 1/2 full $\rightarrow$ no resize (comfortable) • too full $\rightarrow$ grow (credit needed).

- Define Φ with $\Phi(A_0) = 0$; after a resize operation the potential is 0.
- Build up potential so that the next expensive operation is prepaid:
 - if the next operation doubles A (i.e., A_i is full), the potential reaches $size_i$;
 - if the next operation halves A (i.e., A_i is $\frac{1}{4}$ full), the potential reaches $size_i/4$.
- With load factor $\alpha_i = num_i/size_i$ (one formula when at least half full, another when less than half full):

$$\Phi(A_i) = \begin{cases} 2 \cdot num_i - size_i & \text{if } \alpha_i \geq \frac{1}{2}, \\ size_i/2 - num_i & \text{if } \alpha_i < \frac{1}{2}. \end{cases}$$

- **Growing direction** ($\alpha \geq 1/2$, $\Phi = 2num - size$): with $size = 8$, potential is 0, 2, 4, 6, 8 for $num = 4, 5, 6, 7, 8$ — it rises as the array approaches full ($\Phi = 8$ when full, ready to pay for copying 8 elements).
- After doubling: $num = 8$, $size = 16$, exactly half full $\rightarrow \Phi = 2(8) - 16 = 0$ — saved credit was used for the resize and savings are back to zero.
- **Shrinking direction** ($\alpha < 1/2$, $\Phi = size/2 - num$): with $size = 16$, potential is 0, 1, 2, 3, 4 for $num = 8, 7, 6, 5, 4$ — the emptier the array, the more credit saved to pay for shrinking (near 1/4 full, $\Phi = 4 = size/4$).
- Battery picture: potential is smallest around 50% full and grows as we approach a resize (toward 100%: save for growing; toward 25%: save for shrinking). Think of it as a savings account: pushes/pops deposit, a resize withdraws.
- Exam cheat sheet: num = elements, $size$ = capacity, $\alpha = num/size$ = how full, Φ = saved credit; near 100% $\rightarrow$ save for growing; near 25% $\rightarrow$ save for shrinking; after a resize credit drops back down; result: push and pop are $O(1)$ amortized.

Slide 22: Amortized cost: case analysis

We must show that $\hat{c}_i \in O(1)$ in each case — i.e., individual operations can cost $O(n)$ (copying during resize), but resizes don't happen often, so the *amortized* cost of each push or pop is constant:

- **Case 1, cheap push:** there is room $\rightarrow$ just insert, no resize (analyze for $\alpha_i \geq 1/2$ and $\alpha_i < 1/2$).
- **Case 2, expensive push:** array full $\rightarrow$ double, copy, insert — saved potential pays for the copying.
- **Case 3, cheap pop:** remove last element, no resize (analyze for $\alpha_i \geq 1/2$ and $\alpha_i < 1/2$).
- **Case 4, expensive pop:** array too empty $\rightarrow$ shrink, copy — saved potential again pays.

(The details were worked out on the board; the two expensive cases are derived on Slides 23–24.)

Slide 23: Amortized cost: expensive push

Push with resize (expensive):

- The array was full: $num_{i-1} = size_{i-1}$ (1)

- We double the array: $size_i = 2 \cdot size_{i-1}$ (2)

- Real cost of copying the array and adding one element: $c_i = num_{i-1} + 1$ (3)

- Potential before:

$$\Phi_{i-1} = 2 num_{i-1} - size_{i-1} = num_{i-1} \quad (\text{using (1)})$$

- Potential after:

$$\Phi_i = 2 num_i - size_i = 2 \quad (\text{using (1), (2), and } num_i = num_{i-1} + 1)$$

- Amortized cost:

$$\hat{c}_i = c_i + \Delta\Phi = num_{i-1} + 1 + (2 - num_{i-1}) = 3.$$

Slide 24: Amortized cost: expensive pop

Pop with resize (expensive):

- The array was $\frac{1}{4}$ full: $4 num_{i-1} = size_{i-1}$ (1)

- We halve the array: $size_{i-1} = 2 \cdot size_i$ (2)

- Real cost of copying the array and popping one element: $c_i = num_{i-1} + 1$ (3)

- Potential before:

$$\Phi_{i-1} = size_{i-1}/2 - num_{i-1} = num_{i-1} \quad (\text{using (1)})$$

- Potential after:

$$\Phi_i = size_i/2 - num_i = 1 \quad (\text{using (1), (2), and } num_i = num_{i-1} - 1)$$

- Amortized cost:

$$\hat{c}_i = c_i + \Delta\Phi = num_{i-1} + 1 + (1 - num_{i-1}) = 2.$$

Slide 25: Poll: potential of a given array

Poll (in-class)

What is the potential of the following array? (options: 0, 1, 2, 4)
Forms: <https://forms.office.com/r/e36xVbSVjm>